

Spring MVC — 复习要点

Spring 框架与 Spring MVC 简介 (Framework & MVC Introduction)

Spring Framework 概述

- Spring Framework 是一个为 Java 应用提供全面基础设施支持 (Infrastructure Support) 的 Java 平台。[p.4]
- 核心理念：让开发者专注于业务应用，Spring 处理底层基础设施。[p.4]
- 支持通过 POJO (Plain Old Java Object) 构建应用，非侵入式 (Non-invasive) 地应用企业级服务。[p.4]
- 适用于 Java SE 编程模型以及全部或部分 Java EE 场景。[p.4]
- 创始人：Rod Johnson，Interface21 公司 CEO，曾从事 C/C++ 开发和金融行业。[p.7]
- 2002 年著作 *Expert one-on-one J2EE Development without EJB*，核心信条：“Don't Reinvent the Wheel” (不要重复造轮子)。[p.7]

Spring IoC / AOP 基础回顾

- IoC (Inverse of Control, 控制反转) / Bean 容器：管理对象生命周期与依赖注入。[p.2]
- AOP (Aspect-Oriented Programming, 面向切面编程)：用于权限/安全管理切面、日志管理、事务管理等横切关注点。[p.2]
- 经典三层架构：表示层 (Servlet/JSP)、业务逻辑层 (Service/Business Bean)、模型层 (DAO/Bean)。[p.2]
- Spring MVC 运行于 IoC 容器之上，所有对象 (Controller、Service 等) 均由 IoC 容器管理。[p.8]

Spring MVC 简介

- Spring Web MVC 是 Spring 体系中的轻量级 Web 表示层 (Presentation Layer) 框架。[p.8]
- Spring MVC 的核心是 Controller (控制器)，用于处理请求和响应。[p.8]
- Spring MVC 基于 Spring IoC 容器运行，所有对象被 IoC 容器管理。[p.8]
- Spring 5.x：要求 JDK 8、Servlet 3.1 (Tomcat 8.5+)，支持 JDK 8+ 新特性，支持响应式编程 (Reactive Programming, 事件回调机制)。[p.8]
- Spring 6.x：要求 JDK 17、Servlet 5.0 (Tomcat 10.x)。[p.8]

Spring MVC 架构与核心原理 (Architecture & Core Principles)

请求处理流程 (Request Processing Flow)

- ① 请求首先到达前端控制器 (DispatcherServlet / Front Controller)，委托给具体的控制器处理请求。[p.10]
- ② 前端控制器通过查询处理器映射 (Handler Mapping)，找到 URL 对应的控制器 (Controller)。[p.10]
- ③ 控制器处理请求，包括处理数据、调用业务逻辑 (Business Logic) 等。[p.10]
- ④ 控制器将模型数据 (Model) 打包，连同逻辑视图名 (Logical View Name) 返回给前端控制器。[p.10]
- ⑤ 视图解析器 (View Resolver) 将逻辑视图名匹配成具体的视图实现。[p.10]
- ⑥ 视图 (View) 进行模型数据和视图实现的渲染 (Rendering)。[p.10]
- ⑦ 交付模型数据，给出 Web 响应 (Response)。[p.10]

核心组件 (Core Components)

- **DispatcherServlet (前端控制器/前端调度器)**: 统一入口，所有 HTTP 请求首先到达此处；协调 HandlerMapping、Controller、ViewResolver 等组件协同工作；可在 `web.xml` 中配置，也可通过 Java Config 配置。[p.10, 12]
- **Handler Mapping (处理器映射)**: 根据请求 URL 找到对应的 Controller 及方法。[p.10]
- **Controller (控制器)**: 处理业务逻辑，返回 Model 和逻辑视图名。[p.10]
- **View Resolver (视图解析器)**: 将逻辑视图名解析为具体视图技术 (JSP、FreeMarker、Thymeleaf 等)。[p.10]
- **View (视图)**: 渲染模型数据，生成最终 HTML 响应。[p.10]

与原生 Servlet 对比 (Comparison with Native Servlet)

- 原生 Servlet: 需要手动继承 `HttpServlet`，在 `web.xml` 中逐一配置 URL 映射；处理请求和响应的代码量大、重复度高。[p.15, 17]
- Spring MVC: 通过注解 (`@Controller`、`@GetMapping` 等) 声明式配置，大幅简化开发；自动封装请求参数、自动处理响应类型转换。[p.14–15, 17]

控制器与请求映射 (Controllers & Request Mapping)

Controller 编写 (Writing Controllers)

- **@Controller 注解**: 标识该类为一个 Servlet 控制器，Spring IoC 容器会自动扫描并管理其实例。[p.14]
- **@ResponseBody 注解**: 直接以字符串内容进行响应，不进行页面跳转和视图渲染。[p.14]
- 控制器方法处理请求后，返回模型数据 (Model) 和逻辑视图名，或直接返回响应体。[p.10, 14]

URL 映射注解 (URL Mapping Annotations)

- Spring MVC 通过 URL 映射将 Web 请求的 URL 和 Controller 中的方法进行映射。[p.19]
- URL 映射基于方法 (Method-level mapping)，比传统 `web.xml` 中 Servlet 映射粒度更小、使用更灵活。[p.19]
- **@RequestMapping**: 通用映射，不区分请求方法 (GET/POST/PUT/DELETE)，常用于 Controller

类级别进行全局路径前缀设置。[p.19]

- **@GetMapping**: GET 请求映射, 作用于具体方法。例: `@GetMapping("/hello")` 映射至 `http://localhost:` [p.14, 19]
- **@PostMapping**: POST 请求映射, 作用于具体方法。[p.19]
- **@PutMapping**: PUT 请求映射。[p.19]
- **@DeleteMapping**: DELETE 请求映射。[p.19]

数据绑定与请求参数获取 (Data Binding & Request Parameters)

参数获取原理 (Parameter Acquisition Principle)

- **模型驱动 (Model-Driven)**: Controller 对象构造时, Spring MVC 自动构建一个 Model 对象 (Map 类型), 用于存放请求相关的值, 替代 `request.setAttribute()` 的方式。[p.21]
- 请求到达 Controller 前会经过一系列过滤器/拦截器 (Filters/Interceptors), 实现类型转换 (Type Conversion)、集合取值等功能, 开发者也可对其进行扩展。[p.21]

三种获取参数方式

- **方式一 — 方法参数直接接收**: 使用方法参数直接接收网页属性, 基于规则匹配, 需确保前后端属性名称一致。若不一致, 使用 `@RequestParam` 注解进行参数名映射。[p.21–23]
- **方式二 — JavaBean 封装接收**: 使用 JavaBean 接收封装后的大量数据, 配合 `@RequestBody` 注解, 更为常用, 支持 JSON 数据自动转换。[p.21, 24]
- **方式三 — 路径变量与请求头**: `@PathVariable` 获取请求 URI 中的值; `@RequestHeader` 获取请求头中的值。[p.21]

常用参数注解总结

- **@RequestParam**: 单个请求参数与控制器方法参数的绑定, 可指定参数名映射。[p.21, 23]
- **@RequestBody**: 将 HTTP 请求体 (Body) 绑定到 JavaBean 对象, 常用于 POST/PUT 请求的 JSON 数据接收。[p.21, 24]
- **@PathVariable**: 绑定 URI 模板变量, 用于 RESTful 风格的 URL。[p.21]
- **@RequestHeader**: 绑定请求头中的值。[p.21]
- **@ModelAttribute**: 自定义取值及赋值方法, 可用于数据预初始化。[p.29]

响应处理与视图 (Response Handling & Views)

响应处理方式 (Response Methods)

- Spring MVC 对传统 Servlet 的响应处理进行了大量简化, 提供 `ModelAndView` 对象将响应的内容 (Model) 和视图 (View) 进行解耦合 (Decoupling)。[p.25]
- **@ResponseBody**: 不进行页面跳转和视图渲染, 直接输出响应文本, 包括标准 JSON 格式; 实际使用中一般返回 JSON 字符串。[p.25–26]
- **ModelAndView (模型与视图)**: 通过 JSP、FreeMarker、Thymeleaf 等模板引擎进行视图渲染。[p.25]

- 默认视图采用 JSP 方式；老版本推荐 FreeMarker；3.x 开始推荐 Thymeleaf。[p.25]

ModelAndView 详解

- 产生页面跳转的方法返回类型为 ModelAndView。[p.28]
- ModelAndView 中的对象默认作用域为 request (请求作用域)。[p.28]
- ModelAndView 的页面跳转方式默认为 forward (转发)。[p.28]
- ModelAndView 在 Model 的基础上添加了视图对象，需要程序员手动创建。[p.29]

Model / ModelMap / ModelAndView 对比

- Spring MVC 中用于存放数据的有三个对象：ModelMap、Model、ModelAndView。[p.29]
- ModelMap 和 Model 由拦截器自动创建 (类似于 Struts 2 的值栈概念)，在 Controller 功能方法之前运行。[p.29]
- 可使用 @ModelAttribute 注解自定义取值及赋值方法。[p.29]
- ModelAndView 在 Model 基础上添加了视图对象，需要程序员创建。[p.29]
- 根据继承关系，赋值和取值能力排序：ModelAndView > Model > ModelMap。[p.29]

视图层解决方案 (View Layer Solutions)

- Spring MVC 支持多种 View 层解决方案：JSP (默认)、FreeMarker (老版本推荐)、Thymeleaf (Spring Boot 推荐、当前主流)。[p.30]
- 传统 JSP、FreeMarker 等在组件化 (Componentization) 和去耦合 (Decoupling) 上有明显缺点。[p.30]
- Thymeleaf 的核心特点是数据和 HTML 的分离 (Separation of Data and HTML)，模板文件可以直接在浏览器中预览。[p.30]
- 模板语法对比：[p.31]
 - Velocity: `<p>${message}</p>`
 - FreeMarker: `<p>${message}</p>`
 - Thymeleaf: `<p th:text="${message}">Hello World!</p>`
- Thymeleaf 的优势：静态原型即模板，前端和后端可并行开发。[p.30–32]

Web 容器对象使用 (Web Container Objects)

- 在 Spring MVC 的 Controller 中使用 Web 容器对象 (HttpServletRequest, HttpServletResponse, HttpSession 等)，分为耦合方式和非耦合方式。[p.36]
- 耦合方式 (Coupled)：通过接口在 Controller 方法参数中直接声明并获取容器对象，代码与 Servlet API 紧耦合。[p.36–37]
- 非耦合方式 (Decoupled)：通过 Spring MVC 提供的接口进行注入，降低对 Servlet API 的直接依赖，更有利于单元测试 (可轻松 Mock 容器对象)。[p.36, 38]

参数校验 (Parameter Validation)

@Valid 与 @Validation 注解对比

- @Validation 注解：Spring Framework 提供的验证机制，是 JSR-303 规范的一个变种 (Variant)；可使用在类型、方法和方法参数上，但不能使用在类的成员属性上 (不支持嵌套验证)。[p.33]

- **@Valid 注解**：Hibernate 框架提供的验证机制，符合 JSR-303 标准规范，比 @Validation 更强大；可以使用在类的成员属性上，所以支持嵌套验证 (Nested Validation)。[p.33]
- Spring Framework 默认使用 @Validation 进行参数校验；Spring Boot 同时集成了两种注解，开发者可自行选择。[p.33]

常用验证注解 (Common Validation Annotations)

- **@NotNull**：不能为 null [p.34]
- **@Null**：必须为 null [p.34]
- **@AssertTrue / @AssertFalse**：必须为 true / false [p.34]
- **@Digits**：必须为数字（可指定整数和小数位） [p.34]
- **@Max / @Min**：指定整数的最大值和最小值 [p.34]
- **@Length**：指定字符串的最小和最大长度 [p.34]
- **@NotEmpty**：不能为空（包括 null、空字符串、空集合） [p.34]
- **@NotBlank**：不能为空（包括 null、trim 后为空字符串） [p.34]
- **@Email**：必须为合法的 Email 格式 [p.34]
- **@Pattern**：必须符合指定的正则表达式 [p.34]

拦截器 (Interceptors)

拦截器概述 (Overview)

- Spring MVC 拦截器 (Interceptor) 类似于 Servlet 技术中的过滤器 (Filter)，用于对请求进行前置 (Pre-handle) 和后置 (Post-handle) 的过滤处理。[p.41]
- 实现系统的 plug-in (插件) 功能，达到业务功能部分 (Business Logic) 和非业务功能部分 (Cross-cutting Concerns) 解耦的目的。[p.41]
- Spring MVC 拦截器的实现机制基于 **Spring AOP**，与 Servlet 中的过滤器及其他 Web 框架的过滤/拦截器机制不同。[p.41]

拦截器与过滤器对比 (Interceptor vs Filter)

- **实现机制不同**：拦截器基于 Spring AOP，过滤器基于 Servlet 规范。[p.41-42]
- **作用范围不同**：过滤器作用于所有进入容器的请求 (含静态资源)；拦截器只作用于经过 DispatcherServlet 的 Controller 请求。[p.42]
- **控制粒度不同**：拦截器可以更细粒度地控制，能够获取被拦截方法 (Handler) 的上下文信息 (如目标 Controller、方法参数等)；过滤器只能访问 Request/Response 对象。[p.42]
- **生命周期管理不同**：拦截器由 Spring IoC 容器管理，可以使用依赖注入；过滤器由 Servlet 容器管理。[p.42]

拦截器典型应用场景

- **登录权限检查 (Authentication)**：拦截未登录请求，跳转到登录页。[p.16]
- **日志记录 (Logging)**：记录请求 URL、参数、响应时间、用户信息等。[p.44]
- **性能监控 (Performance Monitoring)**：统计每个请求的执行时间。[p.44]
- **结合 Logback 实现用户流量的监控 (Traffic Monitoring)**。[p.44]

全局异常处理 (Global Exception Handling)

- 通过 `@ControllerAdvice` 注解定义全局异常处理器 (Global Exception Handler)。[p.45]
- 结合 `@ExceptionHandler` 注解处理特定类型的异常。[p.45]
- 避免在每个 Controller 中重复编写 try-catch 逻辑，实现异常处理的统一管理和一致的错误响应格式。[p.45]

登录功能实现示例 (Login Implementation)

- Spring MVC 可实现完整的登录功能：用户提交登录表单 → Controller 接收参数 → 调用 Service 层验证 → 根据结果返回不同视图或 JSON 响应。[p.16]
- 可结合拦截器实现登录状态检查与页面保护 (Authentication Interceptor)。[p.16]

HelloWorld 最小化配置回顾 (HelloWorld Recap)

- Maven 依赖：spring-webmvc。[p.11]
- web.xml 配置：注册 DispatcherServlet，指定配置文件路径。[p.12]
- applicationContext.xml 配置：开启组件扫描 (Component Scan)、配置视图解析器 (View Resolver)。[p.13]
- Controller 编写：@Controller + @GetMapping("/hello") + @ResponseBody。[p.14]
- 运行时访问：http://localhost:8080/hello。[p.14]

本章小结 (Chapter Summary)

- **Spring 框架与 Spring MVC 简介：**Spring Framework 是提供全面基础设施支持的 Java 平台；Spring MVC 是其表示层模块，基于 IoC 容器运行。[p.47]
- **架构和核心原理：**DispatcherServlet (前端控制器) → HandlerMapping (处理器映射) → Controller (控制器) → ModelAndView (模型与视图) → ViewResolver (视图解析) → View 渲染 → Response (响应)。[p.47]
- **使用详解核心技术点：**URL 映射注解 (@RequestMapping / @GetMapping / @PostMapping)、请求参数获取 (@RequestParam / @RequestBody / @PathVariable)、响应处理 (@ResponseBody / ModelAndView)、视图解决方案 (JSP / FreeMarker / Thymeleaf)、参数校验 (@Valid / @Validation)、Web 容器对象使用 (耦合/非耦合方式)。[p.47]
- **扩展学习：**拦截器 (基于 Spring AOP 的请求预处理与后处理)、全局异常处理 (@ControllerAdvice + @ExceptionHandler)、最佳实践问题。[p.47]